

# Documentação do Projeto (v1)

## ESPECIFICAÇÃO DO PROJETO

|                         |                                                                                |
|-------------------------|--------------------------------------------------------------------------------|
| ALUNO                   | Alex Oliveira                                                                  |
| DISCIPLINA              | Computação Aplicada à Educação                                                 |
| ÁREAS DE PESQUISA       | Inteligência Artificial na Educação · Gamificação e Jogos Digitais na Educação |
| VERSÃO DO DOCUMENTO     | 1.1                                                                            |
| DOCUMENTOS RELACIONADOS | Documento de Requisitos · Modelo de Entidades e Relacionamentos                |

## 1. Contexto e motivação

---

### 1.1 O problema educacional

Cursos de computação dedicam a maior parte do esforço a ensinar o aluno a **escrever** código. Ler o código de outra pessoa, entender o que ele faz e descobrir onde ele está errado é uma habilidade tão central quanto — e é treinada quase sempre por acidente, quando o aluno esbarra num defeito no próprio trabalho.

O resultado é previsível: alunos que produzem código razoável travam ao receber um programa alheio com defeito. Falta-lhes um método de busca. Leem o código do começo ao fim esperando que o erro salte aos olhos, em vez de formular hipóteses sobre a natureza do defeito e testá-las.

### 1.2 O objetivo pedagógico

O BugHunter treina **depuração e leitura de código** de forma deliberada, separando explicitamente as duas operações que compõem essa habilidade:

1. **Diagnosticar** — localizar onde está o defeito.
2. **Reparar** — corrigi-lo.

A separação é o mecanismo central do sistema. Ao exigir que o aluno aponte a linha antes de poder editar, o BugHunter impede a estratégia de reescrever a função do zero, que conclui o exercício sem exercitar o diagnóstico.

### 1.3 Público-alvo

Estudantes de graduação em computação cursando disciplinas introdutórias de programação, ou já aprovados nelas e em processo de consolidação. A v1 cobre conteúdo de **Fundamentos de Programação**: variáveis, condicionais, laços, listas e funções.

## 2. Visão geral do produto

---

### 2.1 Em uma página

O aluno entra na plataforma, escolhe um tema e um nível de dificuldade, e recebe um exercício: uma função Python **que já funcionou e foi quebrada de propósito**, com um enunciado descrevendo o que ela deveria fazer e um teste de exemplo mostrando uma entrada e a saída esperada.

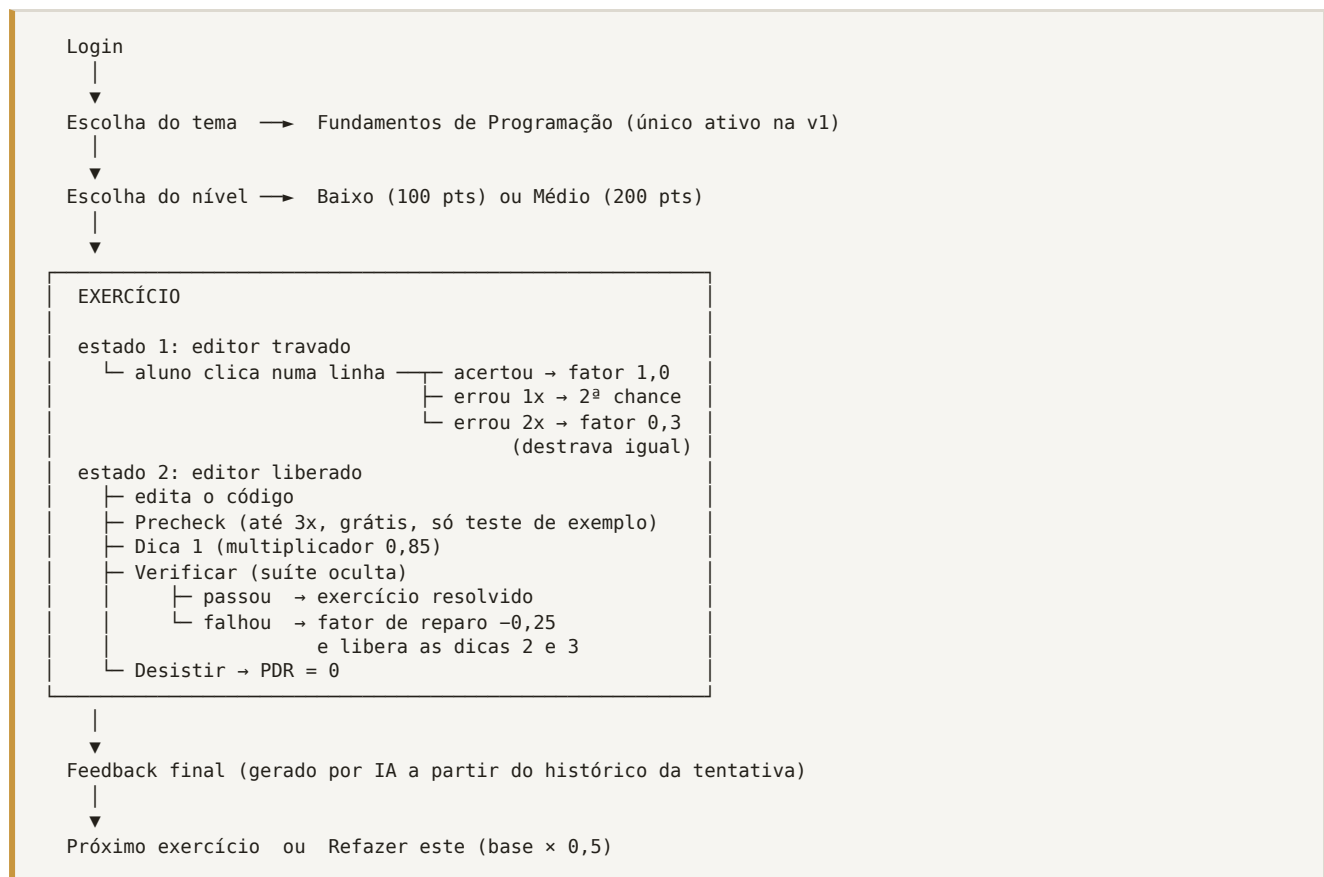
O editor de código começa travado. O aluno precisa primeiro **clicar na linha onde acha que está o defeito**, e tem duas tentativas: acertando na primeira mantém a pontuação cheia, acertando na segunda perde parte dela, e errando as duas o editor destrava mesmo assim com a menor pontuação de localização.

Com o editor liberado, ele corrige o código. Tem dois botões para testar:

- **Precheck** roda apenas o teste de exemplo, aquele visível na tela. Não custa pontos, limitado a três usos.
- **Verificar** roda a suíte completa de testes, que ele nunca viu, incluindo casos de borda. Cada falha reduz a pontuação.

Se travar, pode pedir até três dicas, cada uma revelando mais e custando mais pontos. Se desistir, recebe um feedback explicando qual era o defeito, por que ele passa despercebido e onde o raciocínio dele falhou.

## 2.2 Fluxo completo do usuário



### 3. Escopo da v1

#### 3.1 Dentro e fora

| DIMENSÃO                 | V1                              | FORA DA V1                         |
|--------------------------|---------------------------------|------------------------------------|
| Temas                    | Fundamentos de Programação      | POO, Estrutura de Dados I e II     |
| Níveis                   | Baixo, Médio                    | Alto                               |
| Categorias de defeito    | 4                               | outras 5                           |
| Programas-base           | 5                               | ampliação do catálogo              |
| Exercícios gerados       | 15 a 25                         | —                                  |
| Dicas                    | 12 textos fixos, escritos à mão | dicas geradas sob medida           |
| Uso de IA                | feedback final apenas           | —                                  |
| Pontuação                | PDR por exercício e acumulado   | elo, janela móvel, rankings        |
| Perfil e ranking         | fora                            | telas já desenhadas, entram depois |
| Experimentos de pesquisa | fora                            | —                                  |

#### 3.2 Catálogo de categorias de defeito

Cada categoria pertence a exatamente um nível de dificuldade, e o nível define a pontuação-base do exercício.

| CÓDIGO       | NOME                        | NÍVEL | MUTAÇÃO APLICADA                                                                |
|--------------|-----------------------------|-------|---------------------------------------------------------------------------------|
| CMP_INV      | comparador invertido        | baixo | <code>&lt;</code> ↔ <code>&lt;=</code> , <code>&gt;</code> ↔ <code>&gt;=</code> |
| ARIT_TROC    | operador aritmético trocado | baixo | <code>+</code> ↔ <code>-</code> , <code>*</code> ↔ <code>/</code>               |
| LACO_DESL    | limite de laço deslocado    | médio | <code>range(n)</code> → <code>range(n-1)</code>                                 |
| ACUM_AUSENTE | acumulador não atualizado   | médio | remove a atribuição que atualiza o acumulador                                   |

#### 3.3 Catálogo de programas-base

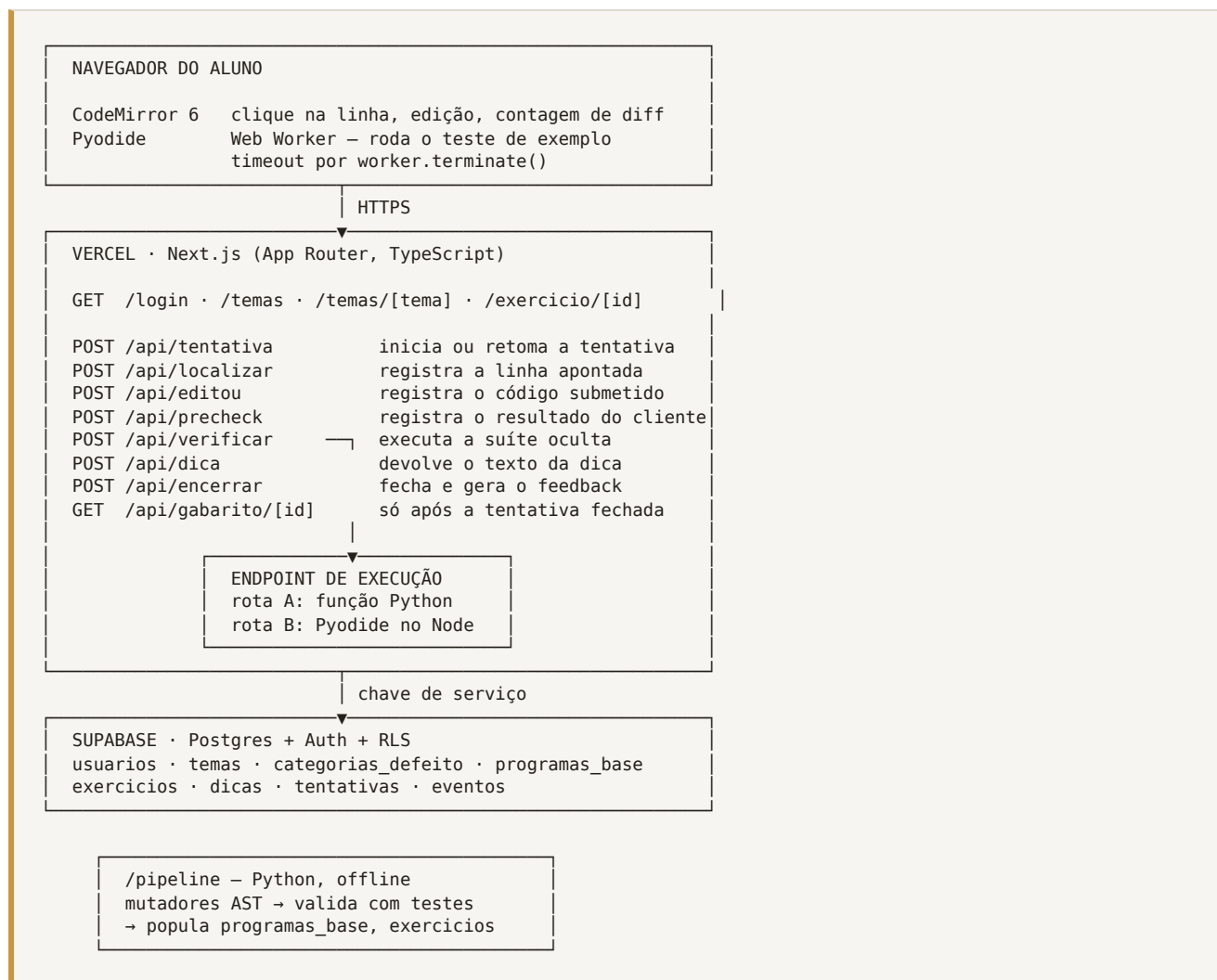
Funções curtas, de 6 a 10 linhas, cada uma acompanhada de um teste de exemplo público e de uma suíte oculta com casos de borda.

| FUNÇÃO                                     | O QUE FAZ                        | CATEGORIAS APLICÁVEIS                                                       |
|--------------------------------------------|----------------------------------|-----------------------------------------------------------------------------|
| <code>media_das_notas(notas)</code>        | média aritmética de uma lista    | todas as 4                                                                  |
| <code>maior_valor(lista)</code>            | maior elemento de uma lista      | <code>CMP_INV</code> , <code>LACO_DESL</code>                               |
| <code>conta_aprovados(notas, corte)</code> | conta quantos passam de um corte | <code>CMP_INV</code> , <code>ARIT_TROC</code> , <code>ACUM_AUSENTE</code>   |
| <code>inverte_lista(lista)</code>          | devolve a lista invertida        | <code>LACO_DESL</code>                                                      |
| <code>soma_pares(numeros)</code>           | soma apenas os números pares     | <code>ARIT_TROC</code> , <code>LACO_DESL</code> , <code>ACUM_AUSENTE</code> |

São **13 combinações** de programa e categoria. Como um mesmo programa pode ter mais de um ponto onde a mesma mutação se aplica — duas comparações, dois operadores aritméticos — e o pipeline gera um exercício por ocorrência (seção 6.2), o número de candidatos fica acima de 13. Descontando as mutações inócuas, a expectativa é de **15 a 25 exercícios válidos**. Se ficar abaixo disso, a correção é acrescentar programas-base, que é barato.

## 4. Arquitetura

### 4.1 Visão de componentes



Toda escrita em **tentativas** e **eventos** passa pelos endpoints, com a chave de serviço. O cliente nunca escreve nessas tabelas diretamente. Isso é o que impede o aluno de forjar um **localizou** correto ou uma pontuação (ver Modelo de Dados, seção 5).

### 4.2 Decisões arquiteturais

**DA-01 · Next.js com TypeScript, não FastAPI com Python.** O aluno já entregou um projeto com Next + Vercel + Supabase. O cliente JavaScript do Supabase é o mais bem documentado da plataforma, e CodeMirror e Pyodide são bibliotecas JavaScript, integradas como componentes naturais em React. O Python do projeto é todo offline e não depende dessa escolha.

**DA-02 · Geração de exercícios por mutação determinística de AST, sem IA.** Mutação sintática garante que a linha do defeito seja conhecida com exatidão — ela é a verdade fundamental contra a qual se avalia o diagnóstico do aluno. Um gerador baseado em modelo de linguagem introduziria incerteza justamente onde não pode haver.

**DA-03 · Precheck no navegador, Verificar no servidor.** O teste de exemplo já está visível na tela, então não há segredo a proteger e ele pode rodar em Pyodide no cliente — resposta instantânea, custo zero, e o isolamento é responsabilidade do navegador. A suíte oculta não pode ir ao cliente, sob pena de ser lida no DevTools, o que anularia a distinção entre os dois botões.

**DA-04 · Dicas escritas à mão por categoria, não geradas em tempo real.** Como cada exercício nasce rotulado com sua categoria, 12 textos fixos cobrem todos os exercícios da v1. Elimina custo por uso, latência, risco de alucinação e a necessidade de guarda-corpos contra vazamento da resposta.

**DA-05 · IA restrita ao feedback final.** É o único ponto do sistema cujo conteúdo depende do que aquele aluno específico digitou. As tentativas erradas possíveis são infinitas, logo o texto não pode ser pré-escrito.

**DA-06 · Eventos como fonte da verdade; pontuação derivada.** Toda ação do aluno vira uma linha imutável na tabela `eventos`, **gravada pelo servidor**. A pontuação de uma tentativa é uma função desses eventos, e o campo `tentativas.pdr_final` é um cache reconstruível. Isso permite auditar a composição de qualquer nota e ajustar os pesos da fórmula sem perder o histórico.

**DA-07 · Segredo do exercício vive no servidor.** Três informações resolvem o exercício e nunca podem chegar ao cliente antes do encerramento: a suíte oculta, o código correto, a linha do defeito. Uma quarta, a **categoria do defeito**, equivale à dica de nível 1 e também fica retida. O cliente recebe apenas o que precisa para exibir a tela.

### 4.3 O risco técnico a resolver antes de tudo

Executar código enviado pelo aluno é executar código não confiável. Há duas rotas possíveis para o endpoint do Verificar, e nenhuma está verificada:

**Rota A — função Python com subprocess.** A Vercel permite misturar runtimes num mesmo projeto. O processo roda com ambiente limpo (`env={}`), para que o código do aluno não consiga ler a chave de serviço do Supabase, com timeout de 5 segundos e diretório temporário.

**Rota B — Pyodide no servidor,** dentro de um route handler do Node. Mantém runtime único e oferece isolamento mais forte: sandbox WASM, sem sistema de arquivos e sem acesso a variáveis de ambiente. O risco é o tempo de carga em partida a frio.

**Plano de ação:** testar as duas em até 4 horas e adotar a que funcionar, preferindo a B. Se ambas falharem, o plano C é executar o Verificar em Pyodide no cliente, buscando a suíte oculta apenas no momento do clique e sem cacheá-la — solução que protege contra o aluno casual, não contra o determinado, e que deve ser declarada como limitação.

## 5. Regras de negócio

### RN-01 • Localização do defeito

O editor de código inicia bloqueado. O aluno dispõe de **duas tentativas de localização**.

| SITUAÇÃO                  | FATOR DE LOCALIZAÇÃO | EFEITO                          |
|---------------------------|----------------------|---------------------------------|
| acertou na 1ª tentativa   | 1,0                  | editor destrava                 |
| errou a 1ª, acertou na 2ª | 0,6                  | editor destrava                 |
| errou as duas             | 0,3                  | editor destrava automaticamente |

A linha correta é `exercicios.linha_defeito`. A comparação é feita **no servidor** — o cliente envia a linha clicada e recebe apenas "correta" ou "incorreta".

### RN-02 • Precheck

Executa exclusivamente o teste de exemplo, no navegador, com limite de 2 segundos. Limitado a **3 usos por tentativa**, contando também as execuções que estouram o tempo. Não altera a pontuação. Após o terceiro uso, o botão fica desabilitado.

### RN-03 • Verificar

Executa a suíte oculta completa, no servidor, com limite de 5 segundos.

- Se todos os testes passam, a tentativa é encerrada com desfecho `resolvido`.
- Caso contrário — teste falho, tempo excedido ou erro de execução — o fator de reparo é **reduzido em 0,25 em termos absolutos**, com piso em 0, e as dicas de nível 2 e 3 são liberadas.

O resultado exibido informa quantos testes passaram do total, **sem revelar o conteúdo dos testes**.

### RN-04 • Dicas

Três níveis, de revelação crescente, obtidos pelo servidor a partir da categoria do exercício.

| NÍVEL | CONTEÚDO                               | MULTIPLICADOR | CONDIÇÃO DE LIBERAÇÃO                            |
|-------|----------------------------------------|---------------|--------------------------------------------------|
| 1     | a natureza do defeito, sem localizá-lo | 0,85          | editor já destravado                             |
| 2     | a região do código                     | 0,65          | dica 1 usada e ao menos um Verificar sem sucesso |
| 3     | quase a correção completa              | 0,40          | dica 2 usada                                     |

As dicas são sequenciais e o multiplicador não é cumulativo — vale o do nível mais alto já usado.

A dica 1 exige o editor destravado porque ela revela a natureza do defeito, o que ajudaria a localizar a linha e contornaria a separação entre diagnóstico e reparo descrita na seção 1.2.

## RN-05 • Pontuação (PDR)

```
PDR = base × (0,4 × fator_localização + 0,6 × fator_reparo)
      × multiplicador_dica × multiplicador_repeticao
```

| COMPONENTE              | ORIGEM                                                                     |
|-------------------------|----------------------------------------------------------------------------|
| base                    | 100 se o nível da categoria é baixo, 200 se é médio                        |
| fator_localização       | eventos <code>localizou</code> , conforme RN-01                            |
| fator_reparo            | inicia em 1,0; -0,25 por evento <code>verificar</code> sem sucesso; piso 0 |
| multiplicador_dica      | maior nível de evento <code>dica</code> , conforme RN-04                   |
| multiplicador_repeticao | 1,0 se <code>numero_tentativa</code> é 1; 0,5 se maior                     |

O resultado é arredondado para o inteiro mais próximo.

**Assinatura da função de cálculo.** Três dos cinco componentes vêm dos eventos; `base` e `multiplicador_repeticao` não. A função é, portanto:

```
calcularPdr(eventos, { base, numero_tentativa }) → inteiro
```

É pura: mesmos argumentos, mesmo resultado, sem consultar o banco. Isso é o que permite recalculer o histórico inteiro quando os pesos mudarem.

**Escopo.** O PDR é local à tentativa. Nenhum exercício anterior entra no cálculo.

**Exemplo trabalhado.** Nível médio, primeira tentativa. O aluno erra a linha 6, acerta a linha 4, usa a dica 1, falha um Verificar e passa no segundo:

```
PDR = 200 × (0,4 × 0,6 + 0,6 × 0,75) × 0,85 × 1,0
     = 200 × (0,24 + 0,45) × 0,85
     = 200 × 0,69 × 0,85
     = 117
```

**PDR acumulado.** Soma dos `pdr_final` das tentativas do aluno, obtida por consulta. Não existe coluna de total acumulado.

## RN-06 • Desistência

Ao desistir, a tentativa é encerrada com desfecho `desistiu` e `pdr_final = 0`. A ação exige confirmação explícita e é irreversível para aquela tentativa.

### RN-07 • Repetição de exercício

Um exercício pode ser tentado quantas vezes o aluno quiser. Toda tentativa após a primeira aplica `multiplicador_repeticao = 0,5`, independentemente do desfecho da anterior.

### RN-08 • Limite de edição

O sistema conta as linhas alteradas em relação ao código original e exibe o contador, com alerta visual acima de 3 linhas. **Na v1 o limite é informativo:** não bloqueia nem penaliza. O número é registrado no evento `editou`.

### RN-09 • Seleção do próximo exercício

Dentro de um tema e nível, na ordem:

1. Se houver tentativa com desfecho `aberto`, retomá-la (RN-10).
2. Senão, o exercício não resolvido de menor `ordem`.
3. Senão, o exercício de menor `ordem`, sujeito a RN-07.

### RN-10 • Tentativa em aberto

Se o aluno fecha a página no meio de uma tentativa, ela permanece com desfecho `aberto`. Ao voltar, a tentativa é retomada no estado reconstruído a partir dos eventos. Não há expiração automática na v1. Só pode existir uma tentativa aberta por aluno e exercício.

### RN-11 • Numeração das tentativas

`numero_tentativa` é atribuído pelo servidor como o maior valor existente para aquele par aluno-exercício, mais um. A atribuição ocorre dentro da transação que cria a tentativa, e a restrição de unicidade no banco é a garantia final contra corrida.

### RN-12 • Registro do código submetido

O evento `editou` é gravado sempre que o aluno aciona Precheck, Verificar ou Encerrar, com o conteúdo atual do editor. Isso garante que sempre exista um código submetido para o feedback final, mesmo quando o aluno edita e desiste sem testar.

## 6. Pipeline de geração de exercícios

---

Roda na máquina do aluno, em Python, na pasta `/pipeline`. Não é publicado.

## 6.1 Algoritmo

```
MUTADOR_VERSAO = "1.0.0"

para cada programa_base P:
    para cada categoria C aplicável a P:

        canonico ← ast.unparse(ast.parse(P.codigo))      # normaliza antes de comparar
        n        ← conta_ocorrencias(canonico, C)        # quantos nós C pode mutar

        para ocorrencia i em 1..n:

            candidato ← ast.unparse(MutadorDe(C, alvo=i).visit(ast.parse(P.codigo)))

            se candidato == canonico:
                descartar                                # mutação não teve efeito

            se não compila(candidato):
                descartar

            se passa(candidato, P.teste_exemplo) é falso:
                descartar                                # violaria o critério 4

            se todos_passam(candidato, P.suite_oculta):
                descartar                                # mutação inócua

            linha ← primeira_linha_divergente(canonico, candidato)
            id    ← uuid5(NAMESPACE, f"{P.id}|{C.codigo}|{i}|{MUTADOR_VERSAO}")

            gravar exercicio(
                id, P.id, C.codigo,
                codigo_com defeito = candidato,
                linha defeito      = linha,
                mutador_versao     = MUTADOR_VERSAO)

atribuir ordem: 1, 2, 3... percorrendo os exercícios gravados
ordenados por (tema, nivel, nome_funcao, categoria, ocorrencia)
```

### Três detalhes que parecem menores e não são:

**A comparação é sempre contra o código canônico**, nunca contra o texto-fonte original.

`ast.unparse` normaliza aspas, parênteses, espaçamento e descarta comentários. Comparar o mutado com o fonte original faria a primeira divergência cair na linha 1 quase sempre — e `linha defeito` é a verdade fundamental do diagnóstico. **É por isso que `programas_base.codigo_correto` armazena a forma canônica**, não o texto como foi digitado.

**A verificação de compilação vem antes de rodar os testes.** Um candidato que nem carrega não tem resultado de teste a consultar.

**O identificador é derivado por `uuid5` de uma chave natural**, não sorteado. Rodar o pipeline duas vezes produz os mesmos identificadores, o que torna a carga idempotente e satisfaz o requisito de reprodutibilidade.

## 6.2 Especificação dos mutadores

Cada mutador recebe um parâmetro `alvo` e altera **exatamente o n-ésimo nó elegível**, deixando os demais intactos. Sem esse parâmetro, um `NodeTransformer` alteraria todas as ocorrências e plantaria vários defeitos no mesmo exercício.

| CÓDIGO       | NÓ ELEGÍVEL                                | TRANSFORMAÇÃO                          |
|--------------|--------------------------------------------|----------------------------------------|
| CMP_INV      | ast.Compare                                | primeiro operador: Lt ↔ LtE , Gt ↔ GtE |
| ARIT_TROC    | ast.BinOp                                  | Add ↔ Sub , Mult ↔ Div                 |
| LACO_DESL    | ast.Call a range                           | subtrai 1 do último argumento          |
| ACUM_AUSENTE | ast.AugAssign ou ast.Assign dentro de laço | remove o nó                            |

Implementação de referência, com a contagem de ocorrências:

```
class InverteComparador(ast.NodeTransformer):
    TROCA = {ast.Lt: ast.LtE, ast.LtE: ast.Lt,
             ast.Gt: ast.GtE, ast.GtE: ast.Gt}

    def __init__(self, alvo):
        self.alvo = alvo      # 1-indexado
        self.vistos = 0

    def visit_Compare(self, node):
        self.generic_visit(node)
        if type(node.ops[0]) in self.TROCA:
            self.vistos += 1
            if self.vistos == self.alvo:
                node.ops[0] = self.TROCA[type(node.ops[0])]()
        return node
```

### 6.3 Critérios de validade

Um candidato só vira exercício se satisfizer **todos** os itens:

1. O código mutado difere do canônico.
2. O código mutado compila.
3. O código mutado **passa no teste de exemplo**. Um exercício resolvível apenas com o Precheck anularia a distinção entre os dois botões. Quando a mutação quebra o teste de exemplo, o candidato é descartado — não se troca o teste, para que todos os exercícios do mesmo programa-base mostrem o mesmo exemplo.
4. O código mutado **falha em ao menos um teste** da suíte oculta.
5. **Exatamente um nó da árvore foi alterado**. O critério é sobre o nó, não sobre linhas de texto: `ACUM_AUSENTE` remove uma linha e desloca todas as seguintes, o que é esperado e não invalida o exercício.

Para `ACUM_AUSENTE`, `linha defeito` aponta a linha **imediatamente anterior** ao ponto da remoção no código mutado, que é onde o aluno precisa olhar para perceber a falta.

## 7. Feedback final gerado por IA

### 7.1 Quando é acionado

Ao encerramento de uma tentativa, tanto por `resolvido` quanto por `desistiu`.

### 7.2 Entrada fornecida ao modelo

| ITEM                      | ORIGEM                                                  |
|---------------------------|---------------------------------------------------------|
| código correto            | <code>programas_base.codigo_correto</code>              |
| código com o defeito      | <code>exercicios.codigo_com_defeito</code>              |
| linha do defeito          | <code>exercicios.linha_defeito</code>                   |
| categoria e sua descrição | <code>categorias_defeito</code>                         |
| código final submetido    | último evento <code>editou</code> (garantido por RN-12) |
| histórico de ações        | eventos da tentativa, em ordem                          |

### 7.3 Saída esperada

Texto em português, em três partes:

1. **Qual era o defeito** — categoria e diferença entre o plantado e o correto.
2. **Por que passa despercebido** — a característica que o torna difícil de ver.
3. **Onde o raciocínio falhou** — análise da tentativa daquele aluno, quando houve correção submetida.

### 7.4 Limites e tratamento de falha

O texto é gerado uma vez, no encerramento, e **armazenado** em `tentativas.feedback_texto`. Não é regerado a cada visualização.

Se a chamada falhar ou exceder 15 segundos, o sistema exibe um feedback de reserva montado com os dados estruturados — categoria, `descricao_curta`, e o diff entre plantado e correto — sem análise da tentativa. **O encerramento nunca é bloqueado por falha na geração.**

## 8. Telas

| # | TELA                              | PROPÓSITO                   | V1  |
|---|-----------------------------------|-----------------------------|-----|
| 1 | Login                             | autenticação                | sim |
| 2 | Escolha do tema                   | seleção de tema             | sim |
| 3 | Nível de dificuldade              | seleção de nível            | sim |
| 4 | Exercício — editor travado        | etapa de localização        | sim |
| 5 | Exercício — editor liberado       | edição e testes             | sim |
| 6 | Exercício — resultado do precheck | retorno do teste de exemplo | sim |
| 7 | Feedback final                    | encerramento da tentativa   | sim |
| 8 | Progresso por categoria           | desempenho do aluno         | não |
| 9 | Ranking semanal e elo             | comparação entre alunos     | não |

As telas 4, 5 e 6 são a **mesma rota** em estados diferentes, não três páginas distintas.

## 9. Decisões tomadas na redação desta documentação

Pontos que não haviam sido discutidos e foram resolvidos aqui. **Merecem validação antes da implementação.**

| #    | PONTO                                  | DECISÃO                                          | ALTERNATIVA DESCARTADA                  |
|------|----------------------------------------|--------------------------------------------------|-----------------------------------------|
| D-01 | Tentativas de localização              | duas; na segunda falha destrava com fator 0,3    | três cliques                            |
| D-02 | Limite de linhas editadas              | informativo, sem bloqueio nem penalidade         | bloquear ou penalizar                   |
| D-03 | Repetição de exercício                 | ilimitada, multiplicador 0,5 fixo                | decaimento progressivo                  |
| D-04 | Ordem dos exercícios                   | fixa, atribuída pelo pipeline                    | aleatória ou adaptativa                 |
| D-05 | Momento do feedback                    | ao resolver e ao desistir                        | só ao desistir                          |
| D-06 | Persistência do feedback               | gerado uma vez e armazenado                      | regerado a cada visita                  |
| D-07 | Confirmação de e-mail                  | desativada na v1                                 | exigir confirmação                      |
| D-08 | Dica 1 com editor travado              | <b>não liberada</b> ; exige localização feita    | sempre disponível                       |
| D-09 | Verificar com tempo excedido           | conta como sem sucesso: -0,25 e libera dicas     | não penalizar                           |
| D-10 | Precheck com tempo excedido            | consome um dos 3 usos                            | não consumir                            |
| D-11 | Mutação por ocorrência                 | um exercício por nó elegível                     | um exercício por par programa-categoria |
| D-12 | Teste de exemplo quebrado pela mutação | descarta o candidato                             | trocar o teste de exemplo               |
| D-13 | Identificadores dos exercícios         | <code>uuid5</code> de chave natural, idempotente | <code>gen_random_uuid()</code>          |
| D-14 | Programas-base                         | entidade própria, com o tema                     | replicados em cada exercício            |
| D-15 | Evento de encerramento                 | tipo único <code>encerrou</code> com o desfecho  | <code>desistiu</code> mais inferência   |

## 10. Riscos

| #    | RISCO                                                    | PROB. | IMPACTO | MITIGAÇÃO                                                          |
|------|----------------------------------------------------------|-------|---------|--------------------------------------------------------------------|
| R-01 | Nenhuma rota de execução do Verificar funciona na Vercel | média | alto    | spike na primeira semana; plano C no cliente                       |
| R-02 | Mutações geram poucos exercícios válidos                 | média | médio   | acrescentar programas-base                                         |
| R-03 | Supabase pausado por inatividade                         | alta  | baixo   | cron diário no GitHub Actions                                      |
| R-04 | Feedback da IA impreciso                                 | média | médio   | dados estruturados no prompt; reserva; revisão dos primeiros casos |
| R-05 | Partida a frio do Pyodide no servidor                    | média | médio   | medir no spike; usar a rota A se inviável                          |
| R-06 | Escopo volta a crescer                                   | alta  | alto    | este documento define o escopo                                     |

## 11. Glossário

| TERMO                | SIGNIFICADO                                                                                                   |
|----------------------|---------------------------------------------------------------------------------------------------------------|
| AST                  | <i>Abstract Syntax Tree</i> . Representação em árvore da estrutura de um programa.                            |
| Categoria de defeito | Classificação do tipo de erro plantado. Define o nível e as dicas.                                            |
| Código canônico      | Saída de <code>ast.unparse</code> sobre o programa correto. É a forma armazenada e a base de toda comparação. |
| Desfecho             | Como uma tentativa terminou: <code>resolvido</code> , <code>desistiu</code> ou <code>aberto</code> .          |
| Evento               | Registro imutável de uma ação do aluno, gravado pelo servidor.                                                |
| Mutador              | Rotina que altera um nó da AST para plantar um defeito de categoria específica.                               |
| Ocorrência           | Índice do nó elegível dentro do programa. Um exercício por ocorrência.                                        |
| PDR do exercício     | Pontos obtidos numa tentativa. Teto de 100 no nível baixo, 200 no médio.                                      |
| PDR acumulado        | Soma dos PDR das tentativas do aluno. Sem teto, não armazenado.                                               |
| Precheck             | Execução do teste de exemplo, no navegador, sem custo de pontos.                                              |
| Programa-base        | Função Python correta, com teste de exemplo e suíte oculta, da qual os exercícios derivam.                    |
| Pyodide              | Python compilado para WebAssembly, executável no navegador.                                                   |
| Suíte oculta         | Conjunto completo de testes, com casos de borda, que o aluno nunca vê.                                        |
| Tentativa            | Uma passagem do aluno por um exercício, do início ao encerramento.                                            |
| Verificar            | Execução da suíte oculta, no servidor, com custo de pontos em caso de insucesso.                              |